

# 20 Preguntas y Respuestas sobre AJAX en JavaScript (didáctico)

## Punto 1: Qué es AJAX y para qué sirve

### 1. ¿Qué es AJAX y cuál es su utilidad?

**Respuesta:** AJAX (Asynchronous JavaScript And XML) permite comunicarse con el servidor **sin recargar la página**, enviando y recibiendo datos de forma asíncrona.

**Ejemplo:**

```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'http://localhost:3000/api/productos', true);
xhr.onload = function() {
    console.log(xhr.responseText); // Muestra los datos del servidor
};
xhr.send();
```

### 2. ¿Cuál es la diferencia entre AJAX y un formulario tradicional?

**Respuesta:**

- Formulario tradicional: envía datos y recarga la página.
- AJAX: envía/recibe datos sin recargar la página.

## Punto 2: Comunicación cliente-servidor con AJAX

### 3. ¿Cómo realizar una petición GET simple al servidor con XMLHttpRequest?

```
const xhr = new XMLHttpRequest();
xhr.open('GET', '/api/productos', true);
xhr.onload = () => console.log(xhr.responseText);
xhr.send();
```

### 4. ¿Cómo se realiza una petición POST para enviar datos al servidor?

```
const xhr = new XMLHttpRequest();
xhr.open('POST', '/api/contacto', true);
xhr.setRequestHeader('Content-Type', 'application/json');
xhr.onload = () => console.log(xhr.responseText);
xhr.send(JSON.stringify({ nombre: "Ana", mensaje: "Hola" }));
```

### 5. ¿Qué significa que AJAX sea asíncrono?

**Respuesta:** Que la página no se bloquea mientras se espera la respuesta del servidor. Puedes seguir interactuando con la página.

## Punto 3: Objeto XMLHttpRequest y sus propiedades/métodos

### 6. ¿Qué métodos principales tiene XMLHttpRequest?

- `open(method, url, async)` → inicializa la petición.
- `send(data)` → envía la petición.
- `setRequestHeader(header, value)` → agrega cabeceras.

### 7. ¿Qué propiedades usamos para manejar la respuesta?

- `status` → código HTTP de la respuesta.
- `responseText` → datos como texto.
- `readyState` → estado de la petición (0 a 4).

## Punto 4: Recepción y procesamiento de la respuesta

### 8. ¿Cómo procesar una respuesta en JSON?

```
xhr.onload = function() {  
  if(xhr.status === 200){  
    const data = JSON.parse(xhr.responseText);  
    console.log(data); // Array o objeto recibido  
  }  
};
```

### 9. ¿Cómo manejar errores en AJAX?

```
xhr.onerror = function() {  
  console.log('Error en la petición AJAX');  
};
```

### 10. ¿Cómo mostrar los datos recibidos en la tabla HTML?

```
const tabla = document.getElementById('tabla');  
xhr.onload = function() {  
  const data = JSON.parse(xhr.responseText);  
  tabla.innerHTML = '';  
  data.forEach(item => {  
    const row = document.createElement('tr');  
    row.innerHTML = ` ${item.id}</td><td>${item.nombre}</td>`;     tabla.appendChild(row);   }); }; |
```

## Punto 5: Envío de datos al servidor

### 11. ¿Cómo enviar datos usando GET con parámetros?

```
const categoria = "Electrónica";
const xhr = new XMLHttpRequest();
xhr.open('GET', `/api/productos?categoria=${categoria}`, true);
xhr.onload = () => console.log(xhr.responseText);
xhr.send();
```

### 12. ¿Cómo enviar datos usando POST en formato JSON?

```
const datos = { nombre: "Juan", mensaje: "Hola" };
const xhr = new XMLHttpRequest();
xhr.open('POST', '/api/contacto', true);
xhr.setRequestHeader('Content-Type', 'application/json');
xhr.onload = () => console.log(xhr.responseText);
xhr.send(JSON.stringify(datos));
```

### 13. ¿Cómo enviar los datos de un formulario HTML usando AJAX?

```
const form = document.getElementById('formulario');
form.addEventListener('submit', e => {
  e.preventDefault();
  const datos = {
    nombre: form.nombre.value,
    email: form.email.value
  };
  const xhr = new XMLHttpRequest();
  xhr.open('POST', '/api/contacto', true);
  xhr.setRequestHeader('Content-Type', 'application/json');
  xhr.onload = () => console.log(xhr.responseText);
  xhr.send(JSON.stringify(datos));
});
```

## Punto 6: Formatos de recepción de datos

### 14. ¿Qué formatos puede recibir AJAX?

- text/html → HTML
- application/json → JSON
- application/xml → XML

### 15. Ejemplo de recepción de JSON y HTML:

```
// JSON
xhr.onload = () => {
  const data = JSON.parse(xhr.responseText);
  console.log(data);
};
// HTML
xhr.onload = () => {
  document.getElementById('div').innerHTML = xhr.responseText;
};
```

## Punto 7: Uso práctico y APIs públicas

### 16. ¿Cómo usar AJAX para consumir una API pública que devuelve JSON?

```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://api.coindesk.com/v1/bpi/currentprice.json',
true);
xhr.onload = function() {
    const data = JSON.parse(xhr.responseText);
    console.log("Precio Bitcoin:", data.bpi.USD.rate);
};
xhr.send();
```

### 17. Ejemplo: filtrar datos de la API antes de mostrar:

```
xhr.onload = function() {
    const data = JSON.parse(xhr.responseText);
    console.log("Precio en USD:", data.bpi.USD.rate);
};
```

### 18. ¿Cómo usar AJAX para actualizar la página sin recargar?

- Recibir los datos con `onload` y actualizar elementos del DOM dinámicamente, como tablas o listas.

```
const lista = document.getElementById('lista');
xhr.onload = () => {
    const data = JSON.parse(xhr.responseText);
    lista.innerHTML = '';
    data.forEach(item => {
        const li = document.createElement('li');
        li.textContent = item.nombre;
        lista.appendChild(li);
    });
};
```

### 19. ¿Cómo usar parámetros en GET y POST juntos?

- GET → para filtrar o solicitar datos
- POST → para enviar datos o formularios

```
// GET con filtro
xhr.open('GET', '/api/productos?categoria=Electrónica', true);
xhr.send();
```

```
// POST para formulario
xhr.open('POST', '/api/contacto', true);
xhr.setRequestHeader('Content-Type', 'application/json');
xhr.send(JSON.stringify({ nombre: "Ana", mensaje: "Hola" }));
```

### 20. Buenas prácticas al usar AJAX

- Siempre usar `onerror` para capturar errores.

- Usar `JSON.parse` para datos JSON.
- Validar datos en el cliente y servidor.
- Evitar recargar la página innecesariamente.

Con estas **20 preguntas y respuestas con ejemplos claros**, los alumnos podrán:

- Comprender qué es AJAX.
- Usar `XMLHttpRequest` correctamente.
- Enviar y recibir datos JSON.
- Actualizar dinámicamente el DOM.
- Evitar errores comunes en la implementación.